

1. UserService (회원 관련 기능)

회원가입

- 메서드명: `public boolean register(UserRegisterDTO dto)`
- 입력 데이터 (DTO): `String loginId, String password, String name`
- 출력 데이터: `boolean` (성공 시 `true`, 아이디 중복 시 `false`)
- 설명: 사용자가 입력한 정보로 중복 검사 후 `User` 테이블에 데이터를 생성합니다. 기본 권한(`role`)은 'USER'로 저장됩니다.

로그인

- 메서드명: `public User login(String loginId, String password)`
 - 입력 데이터: `String loginId, String password`
 - 출력 데이터: `User` (성공 시 로그인한 유저의 전체 정보 객체, 실패 시 `null`)
 - 설명: 아이디와 비밀번호를 검증하여 일치하면 유저 객체를 반환합니다. View단은 반환된 유저의 `role` 을 확인하여 일반 회원 화면 또는 관리자 화면으로 분기합니다.
-

2. ProductService (상품 및 관리자 상품 관리 기능)

상품 목록 조회

- 메서드명: `public List<Product> getProductList()`
- 입력 데이터: 없음
- 출력 데이터: `List<Product>` (전체 상품 리스트)
- 설명: 쇼핑몰 메인 화면에 보여줄 전체 상품 목록을 조회합니다.

상품 상세 조회

- 메서드명: `public Product getProductDetail(int productId)`
- 입력 데이터: `int productId`
- 출력 데이터: `Product` (특정 상품 객체)
- 설명: 선택한 특정 상품의 이름, 가격, 남은 재고 등 상세 정보를 조회합니다.

[관리자] 상품 등록

- 메서드명: `public boolean addProduct(ProductAdminDTO dto)`
- 입력 데이터 (DTO): `String productName, int price, int stock`

- 출력 데이터: boolean (성공 여부)
- 설명: 관리자가 새로운 상품을 등록하는 기능입니다.

[관리자] 상품 정보 수정

- 메서드명: `public boolean updateProduct(Product updateData)`
 - 입력 데이터: Product (수정할 상품의 id와 변경될 데이터가 담긴 객체)
 - 출력 데이터: boolean (성공 여부)
 - 설명: 관리자가 기존 상품의 이름, 가격, 재고 수량을 수정합니다.
-

3. OrderService (주문 및 관리자 주문 관리 기능)

여러 상품 동시 주문

- 메서드명: `public boolean createOrder(int userId, List<OrderItemDTO> orderItems)`
- 입력 데이터: int userId, List<OrderItemDTO> (각 상품의 productId와 quantity가 담긴 리스트)
- 출력 데이터: boolean (주문 성공 시 true, 재고 부족 등 실패 시 false)
- 설명: 1. Orders 테이블에 주문 기본 정보를 먼저 생성합니다. 2. 선택한 상품 종류만큼 Order_Item 테이블에 데이터를 각각 나누어 넣을 때, 현재 시점의 상품 가격을 조회하여 order_price에 함께 기록합니다. 3. 주문 수량만큼 Product 테이블의 stock을 차감합니다. (이 과정은 하나의 트랜잭션으로 묶여야 합니다.)

회원 주문 내역 확인

- 메서드명: `public List<OrderResponseDTO> getUserOrderList(int userId)`
- 입력 데이터: int userId
- 출력 데이터: List<OrderResponseDTO> (주문 날짜, 상품 정보들, 총 결제 금액, 현재 주문 상태 등이 포함된 커스텀 DTO 리스트)
- 설명: 로그인한 회원의 주문 이력을 확인하기 위해 Orders, Order_Item, Product 테이블을 조인하여 데이터를 가져옵니다.

주문 취소

- 메서드명: `public boolean cancelOrder(int orderId)`
- 입력 데이터: int orderId
- 출력 데이터: boolean (성공 여부)

- **설명:** 주문을 취소하면 `orders` 테이블의 상태를 '취소'로 바꾸고, 해당 주문에 포함되었던 상품들의 수량을 `Product` 테이블의 `stock` 에 다시 더해줍니다.

[관리자] 전체 주문 조회

- **메서드명:** `public List<OrderResponseDTO> getAllOrderList()`
- **입력 데이터:** 없음
- **출력 데이터:** `List<OrderResponseDTO>` (시스템의 모든 주문 리스트)
- **설명:** 관리자가 모든 회원의 주문 내역과 진행 상태를 한눈에 확인할 수 있도록 전체 데이터를 조회합니다.

[관리자] 주문 상태 관리

- **메서드명:** `public boolean updateOrderStatus(int orderId, String status)`
- **입력 데이터:** `int orderId, String status` (예: "배송중", "배송완료")
- **출력 데이터:** `boolean` (성공 여부)
- **설명:** 관리자가 특정 주문 건에 대해 배송 상태 등을 변경합니다.

💡 협업 꿀팁

- **Model(Domain) 객체 공동 사용:** 위 명세서에 등장하는 `User`, `Product` 같은 핵심 도메인 객체들과 DTO들은 두 사람이 공통으로 사용하므로, 패키지 생성 시 가장 먼저 구조를 정의해 두고 공유하는 것이 좋습니다.
- **인터페이스 선언 먼저 하기:** 자바의 `interface` 문법을 활용해 `public interface OrderService` 형태로 캡데기 메서드만 미리 선언해 두면, 알맹이 코드가 완성되지 않아도 상대방이 내 메서드를 호출하는 UI 코드를 미리 작성할 수 있습니다.